

Problem Set 2

Harvard SEAS - Fall 2026

Due: Thursday 2026-09-24 (11:59pm)

Please see the syllabus for the full collaboration and generative AI policy, as well as information on grading, late days, and revisions.

All sources of ideas, including (but not restricted to) any collaborators, AI tools, people outside of the course, websites, ARC tutors, and textbooks other than Hesterberg–Vadhan must be listed on your submitted homework along with a brief description of how they influenced your work. You need not cite core course resources, which are lectures, the Hesterberg–Vadhan textbook, sections, SREs, problem sets and solutions sets from earlier in the semester. If you use any concepts, terminology, or problem-solving approaches not covered in the course material by that point in the semester, you must describe the source of that idea. If you credit an AI tool for a particular idea, then you should also provide a primary source that corroborates it. Github Copilot and similar tools should be turned off when working on programming assignments.

If you did not have any collaborators or external resources, please write 'none.'

Your name: Andrew Jing

Collaborators and External Resources: Claude code turned my algorithm pseudocode, photos and python, into latex. Also reviewed my write-ups for grammar, latex typing mistakes, etc.

No. of late days used on previous psets: 1

No. of late days used after including this pset: 1

1. (designing reductions) The purpose of this exercise is to give you practice designing reductions and proving their correctness and runtime.

Consider the following computational problem:

Input: Points $(x_0, y_0), \dots, (x_{n-1}, y_{n-1})$ in the $\mathbb{R}^2$ plane that are the vertices of a convex polygon (in an arbitrary order) whose interior contains the point $(1, 1)$.

Output: The area of the polygon formed by the points

Computational Problem AreaOfConvexPolygon()

- (a) Show that $\text{AreaOfConvexPolygon} \leq_{O(n),n} \text{Sorting}$. Be sure to analyze both the correctness and runtime of your reduction. *Maximum 25 sentences, around one page*

In this part and the next one, you may assume that a point $(x, y) \in \mathbb{R}^2$ can be converted into polar coordinates (r, θ) about $(1, 1)$ in constant time.

You may find the following facts useful and you may use them (as well as other facts from geometry) without proof:

- The polar coordinates (r, θ) of a point (x, y) about $(1, 1)$ are the unique real numbers $r \geq 0$ and $\theta \in [0, 2\pi)$ such that $x - 1 = r \cos \theta$ and $y - 1 = r \sin \theta$. Or, more geometrically, $r = \sqrt{(x - 1)^2 + (y - 1)^2}$ is the distance of the point from $(1, 1)$, and θ is the angle between the positive x -axis and the ray from $(1, 1)$ to the point.
- The area of a triangle is $A = \sqrt{s(s - a)(s - b)(s - c)}$ where a, b, c are the side lengths of the triangle and $s = \frac{a+b+c}{2}$ ([Heron's Formula](#)).

PolarSortArea $((x_0, y_0), \dots, (x_{n-1}, y_{n-1}))$

Input : Points $(x_0, y_0), \dots, (x_{n-1}, y_{n-1})$, the vertices of a convex polygon containing $(1, 1)$ in its interior (so each $(x_i, y_i) \neq (1, 1)$)

Output : The area of the polygon

```

1 foreach  $i \in [n]$  do
2    $(r_i, \theta_i) = \text{PolarConvert}((x_i, y_i));$ 
3  $((\theta'_0, (r'_0, x'_0, y'_0)), \dots, (\theta'_{n-1}, (r'_{n-1}, x'_{n-1}, y'_{n-1}))) =$ 
    $\text{SortingOracle}(((\theta_0, (r_0, x_0, y_0)), \dots, (\theta_{n-1}, (r_{n-1}, x_{n-1}, y_{n-1})))));$ 
4  $sum = 0;$ 
5 foreach  $i = 1, \dots, n - 1$  do
6    $a = \sqrt{(x'_{i-1} - x'_i)^2 + (y'_{i-1} - y'_i)^2};$ 
7    $s = \frac{1}{2}(a + r'_{i-1} + r'_i);$ 
8    $sum = sum + \sqrt{s(s - a)(s - r'_{i-1})(s - r'_i)};$ 
9  $a = \sqrt{(x'_0 - x'_{n-1})^2 + (y'_0 - y'_{n-1})^2};$ 
10  $s = \frac{1}{2}(a + r'_0 + r'_{n-1});$ 
11  $sum = sum + \sqrt{s(s - a)(s - r'_0)(s - r'_{n-1})};$ 
12 return  $sum$ 
```

Algorithm 0.1: PolarSortArea

As an overview of what the algorithm does, it (1) sorts all points by polar angle from (1,1) and (2) calculates the area of the triangle formed by (1,1) and each pair of neighboring vertices, and (3) adds those triangles' areas for the total.

The algorithm is correct because in a convex polygon, triangles made by a fixed interior point and two neighboring vertices on the polygon never overlap, but all of them together cover the entire convex polygon, so the sum of the triangles' areas is exactly the area of the entire convex polygon. We can find pairs of neighboring vertices by sorting their polar angle relative to the interior point, and we know that this sorted vector has no duplicate keys (angles). This is because (1) no vertex can be (1,1) since that point is interior, so each vertex has only 1 valid θ , (2) each vertex is unique or else the sides between repeated vertices would be length 0 and cannot be part of the convex polygon (or if we allow dupes they only add triangles of area 0 anyway), and (3) two distinct vertices with the same θ to an interior point cannot be a part of a convex polygon, since the polygon's outline would need to bend outwards to keep (1,1) inside. So all θ_i are distinct, and we know that in the sorted array two neighboring entries (or the first and last entries, whose side straddles $\theta = 0$) form pairs of neighboring vertices, or else we would break convexity. So for each neighboring pair, the triangle it forms with (1,1) has two sides that by definition are the polar radius from (1,1) and we calculate the remaining side opposite (1,1) by keeping non-polar coordinates together with their polar representations. Also, there are at least 3 points in a polygon, so neither the loop nor non-loop area calculations get skipped. Since this method retains all sorted points as a permutation of the original input, and because these partial triangles add to exactly the full polygon, the accumulated *sum* across the partial triangles' areas are indeed the correct answer.

Let's look at runtime. The line 1 loop is n iterations times a constant-time conversion for $O(n)$. Constructing the input for *SortingOracle* is $O(n)$ copies, while the oracle itself is 1 step. Initializing *sum* is one step. Looking at lines 5-11 as one loop, we perform n iterations of the same set of math operations for a total of $O(n)$ time (assuming these operations are constant relative to number magnitude). Then, the return statement is also 1 step. In total, we have an algorithm that reduces to Sorting in $O(n) + O(n) + 1 + 1 + O(n) + 1 = O(n)$ time. Also, we call *SortingOracle* exactly once, with n entries to sort. So $\text{AreaOfConvexPolygon} \leq_{O(n),n} \text{Sorting}$ (more specifically $\text{AreaOfConvexPolygon} \leq_{O(n),1 \otimes n} \text{Sorting}$).

- (b) Deduce that $\text{AreaOfConvexPolygon}$ can be solved in time $O(n \log n)$. *Maximum 4-5 sentences*

Since $\text{AreaOfConvexPolygon} \leq_{O(n),1 \otimes n} \text{Sorting}$, and because *MergeSort* solves *Sorting* in $O(n \log n)$, then we know it's possible to solve $\text{AreaOfConvexPolygon}$ in $O(n) + O(1 \cdot O(n \log n)) = O(n + n \log n) = O(n \log n)$. (See Lemma 4.3 and 4.4).

- (c) (*challenge; extra credit; optional¹) Come up with a way to avoid conversion to polar coordinates and any other trigonometric functions in solving $\text{AreaOfConvexPolygon}$ in

¹This problem is meant to be done based on your enjoyment/interest and only if you have time. It won't make a difference between N, L, R-, and R grades (meaning it will only impact whether an R gets increased to an R+), and course staff will deprioritize questions about this problem at office hours and on Ed.

time $O(n \log n)$. Specifically, design an $O(n)$ -time reduction that makes $O(1)$ calls to a *Sorting oracle* on arrays of length at most n , using only arithmetic operations $+$, $-$, $\times$, $\div$, and $\sqrt{}$, along with comparators like $<$ and $=$. (Hint: first partition the input points according to which quadrant they belong in, and consider the slope of the line from a vertex (x,y) to the origin.)

Instead of converting to polar, we can split the points into 4 sub-groups, one for each quadrant relative to $(1,1)$ (let's say the upper right and bottom left quadrants own the axes relative to $(1,1)$). Let's sort (with *SortingOracle*) each of the 4 quadrants by the slope of the lines between each vertex and $(1,1)$, defining vertical slopes as ∞ (or just place them at the end of the sorted subgroups). By the same logic as with polar conversion, within these quadrants, the vertices' slopes are unique. Then concatenate the subgroups in counter-clockwise order, and follow a similar area calculation scheme as in 1a (shifting the radius calculation to this step instead of earlier during conversion). By similar logic as 1a, this non-trigonometric algorithm is correct, with these necessary addendums: the first point in a non-empty sorted quadrant is a neighboring vertex to the last point in the closest (clockwise) non-empty sorted quadrant (and the opposite for the last point in each non-empty quadrant), so the total, concatenated array with 4 sorted subgroups will still guarantee that each neighbor pair by sorting is also a neighbor pair as vertices (and also entries 0 and $n - 1$).

Since each quadrant (relative to $(1,1)$) can't have more points than the entire polygon does, each of the 4 calls to *SortingOracle* is performed on a subgroup of at most n entries. And the reduction algorithm itself performs $O(n)$ steps for slope calculations, $O(n)$ setup of the oracle calls' input arrays, $O(n)$ for concatenation, and $O(n)$ for calculating n partial triangles' areas and adding them together. Plus some other $O(1)$ stuff like returning. So in total $\text{AreaOfConvexPolygon} \leq_{O(n), 4 \otimes n} \text{Sorting}$, which by similar logic to 1b proves there's a way to solve *AreaOfConvexPolygon* in $O(n \log n)$. However, since our reduction doesn't use trigonometry, and a witness *MergeSort* to the sorting oracle also doesn't use trigonometry, we know that this solving in $O(n \log n)$ can happen without trigonometry, either.

Similar techniques to what you are using in this problem are used in algorithms for other important geometric problems, like finding the Convex Hull of a set of points, which has applications in graphics and machine learning.

2. (composition of reductions) The purpose of this exercise is to give you practice in working with the abstract definition of reductions.

Let Π, Γ, Λ be computational problems, and suppose that $\Pi \leq_{T_1(n), q_1(n) \otimes h_1(n)} \Gamma$ and $\Gamma \leq_{T_2(n), q_2(n) \otimes h_2(n)} \Lambda$, for non-decreasing functions $T_1, T_2, q_1, q_2, h_1, h_2$. Determine functions T_3, q_3, h_3 in terms of $T_1, T_2, q_1, q_2, h_1, h_2$ such that

$$\Pi \leq_{O(T_3(n)), q_3(n) \otimes h_3(n)} \Lambda,$$

and justify your answers. You do not need to prove the correctness of the reduction from Π to Λ . Just justify the functions T_3, q_3 and h_3 . You don't need to give a formal proof, *around half a page answer* (Hint: you can take $h_3(n) = h_2(h_1(n))$.)

The reduction from Π to Γ calls a Γ oracle at most $q_1(n)$ times, and if we take the reduction from Γ to Λ as a partial witness to that Γ oracle (which itself calls a Λ oracle at most $q_2(m)$ times each of its $q_1(n)$ instances), then since $m \leq h_1(n)$ and q_2 is non-decreasing the maximum time our reduction from Π to Λ must call the Λ oracle at most $q_3(n) = q_1(n) \cdot q_2(h_1(n))$ times.

Since each Γ oracle has an input of $m \leq h_1(n)$, and since the reduction from Γ to Λ calls the Λ oracle with an input of $p \leq h_2(m)$, because h_2 is non-decreasing then we know $p \leq h_2(m) \leq h_2(h_1(n))$, so $h_3(n) = h_2(h_1(n))$.

Since the Γ to Λ reduction (as partial witness to the oracle) is called at most $q_1(n)$ times with max input size $h_1(n)$, because its runtime is $O(T_2(m))$ where $m \leq h_1(n)$, since T_2 is non-decreasing we know that the total witness runtime (still abstracting the Λ oracle) is at most $q_1(n) \cdot O(T_2(h_1(n)))$. So, accounting for the non-oracle runtime in the reduction from Π to Γ , the total composite reduction's runtime is $T_3 = T_1(n) + q_1(n) \cdot T_2(h_1(n))$. We got rid of the big O in T_3 since it was moved outside of T_3 for total runtime.

3. (augmented binary search trees) The purpose of this problem is to give you experience reasoning about correctness and efficiency of dynamic data-structure operations, on variants of binary-search trees. You may assume that all keys are distinct.

Specifically, we will work with *selection data structures*. We have seen how binary search trees can support `min` queries in time $O(h)$, where h is the height of the tree. A generalization is *selection* queries, where given a natural number q , we want to return the q 'th smallest element of the set. So `DS.select(0)` should return the key-value pair with the minimum key among those stored by the data structure `DS`, `DS.select(1)` should return the one with the second-smallest key, `DS.select(n-1)` should return the one with the maximum key if the set is of size n , and `DS.select((n-1)/2)` should return the median element if n is odd.

It is known that if we *augment* binary search trees by adding to each node v the size of the subtree rooted at v , then `select` queries can be answered in time $O(h)$.²

- (a) In the Github repository, we have given you a Python implementation of size-augmented BSTs supporting search, insertion, and selection, and with a stub for `rotate`. One of the implemented functions (`search`, `insert`, or `select`) has a correctness error, another one is too slow (running in time that's (at least) linear in the number of nodes of the tree rather than linear in the height of tree), and the third is correct. Identify and correct these errors. You should provide a text explanation of the errors and your corrections, as well as implement the corrections in Python.

The `select` function is wrong, because when it prunes the left subtree (and current node) from consideration, it doesn't change what rank to look for in the next recursive step. For the answer to be correct, the `ind` variable needs to be shifted by however many candidates we dropped – so the correction is `return self.right.select(ind - (left_size + 1))`. This balances out how many candidates we drop with what rank we're saying to look for.

The `insert` function takes too long, since it calls `calculate_sizes`, which is $O(n)$ instead of $O(h)$. Instead, we only have to recalculate the sizes that were affected by the insert, which is the chain of all ancestors of the new node. So we can just increment the sizes of all nodes we visit (except the inserted node, which already starts at size 1) and not call `calculate_sizes`.

`search` is fine.

- (b) Describe (in pseudocode or pictures) how to extend `rotate` to size-augmented BSTs, and argue that your extension maintains the runtime $O(1)$. Explain the intuition *in 2-3 sentences* that your new rotation operation preserves the invariant of correct size-augmentations. (That is, if every node's size attribute had the correct subtree size before

²Note that the Roughgarden text uses a different indexing than us for the inputs to `select`. For Roughgarden, the minimum key is selected by `select(1)`, whereas for us it is selected by `select(0)`.

the operation, then the same is true after the operation.)

```
RotateRight(y, p)
Input           : A node  $y$  of a size-augmented BST, and its parent  $p$  (None if  $y$  is the
                  root)
Output          : The new root  $x$  of the rotated subtree (None if  $y$  has no left child)
1 if  $y.left = \text{None}$  then
2   | return None
3  $x = y.left$ ;
4  $y.left = x.right$ ;
5  $x.right = y$ ;
6  $x.size = y.size$  ;                               /*  $x$  now roots exactly the nodes  $y$  used to */
7  $y.size = 1 + size(y.left) + size(y.right)$  ;      /*  $size(\text{None}) = 0$  */
8 if  $p \neq \text{None}$  then
9   | if  $p.left = y$  then
10  |   |  $p.left = x$ ;
11  | else
12  |   |  $p.right = x$ ;
13 return  $x$ 
```

Algorithm 0.2: RotateRight

The algorithm and justification for a left rotate is symmetrical.

Since subtree size is defined as $1 + size(left) + size(right)$, if we modify a tree, a subtree's size only changes if its own subtrees were changed. Since the algorithm never touches the original $y.right$, $x.left$, and $x.right$ subtrees, those sizes carry over from before the rotate. Only two nodes' subtrees actually change: the original root and its left child, so we calculate the original root's new size by definition, and since its former left child is now the root, we just set it to the old tree size because we haven't inserted or deleted anything.

The runtime is $O(1)$ because we only change pointers, not where the nodes are located, and at maximum we do several assigns, a couple compares, and 2 additions before returning.

Food for thought (do read - it's an important take-away from this problem): This problem concerns size-augmented binary search trees. In lecture, we discussed AVL trees, which are balanced binary search trees where every vertex contains an additional *height* attribute containing the length of the longest path from the vertex to a leaf (height-augmented). Additionally, every pair of siblings in the tree have heights differing by at most 1, so the tree is height-balanced. Note that if we augment a binary search tree both by size (as in the above problem) and by height (and use it to maintain the AVL property), then we create a dynamic data structure able to perform **search**, **insert**, and **select** all in time $O(\log n)$.

4. (reflection) We aim for cs1200 to be a collaborative learning community. Describe two concrete ways in which you have supported, or will try to support, your classmates' learning in the course. Be specific, connecting your answer to the structure of cs1200.

During lecture we often have group activities/breakout discussions. In those moments I try to ask my tablemates what they think and attempt to have all of us participate in the

discussion. Of course, I don't force the point if they don't want to talk. I find this approach to be more chill than everyone doing stuff independently or one person solves and other people listen/look. This applies to CS1200 specifically because it can be jarring to switch from listening to a lecture to remembering to talk, without someone breaking the silence. Also, I'll keep trying to collaborate with classmates to compare approaches to pset problems. The main barrier to this is how busy we all are, but for CS1200 it's probably more important than for other classes, since there can be so many different approaches to one proof.

Quick note on grading: Good responses are usually about a paragraph, with something like 7 or 8 sentences. Most importantly, please make sure your answer is specific to this class and your experiences in it. If your answer could have been edited lightly to apply to another class at Harvard, points will be taken off.

Note: As with the previous psets, you may include your answer in your PDF submission, but the answer should ultimately go into a separate Gradescope submission form.

5. Once you're done with this problem set, please fill out [this survey](#) so that we can gather students' thoughts on the problem set, and the class in general. It's not required, but we really appreciate all responses!